

Ollama + OpenWebUI — Hands-On Setup Walkthrough

From zero to chatting in your browser — with real troubleshooting.



Practical Guide



Docker



OpenWebUI



Troubleshooting

A practical companion to Topics 34–36. Follow it top to bottom and you'll go from nothing installed to chatting with a local model in your browser. **Ends with a real troubleshooting walkthrough** from an actual setup session.

① What is Ollama, simply



One line

Ollama runs AI models on your own computer. No internet, no API key, no bill.

	ChatGPT / Claude API	Ollama
Where the model runs	Someone's server	Your machine
Cost	Per token	Free
Your data	Sent to a company	Never leaves
Needs internet	Yes	Only to download the model once

Two parts to remember

- **The runtime** — the program that loads and runs models. That's what you install.
- **The models** — separate multi-GB downloads (`llama3.2` , `qwen2.5` , `gemma` , etc).



The key mental model

Once running, Ollama is a **server sitting on port 11434**. Everything else — the terminal, OpenWebUI, your Python code — talks to it over HTTP.

② Check it on Docker Hub first

Go to `hub.docker.com` → search `ollama` → open `ollama/ollama` .

What to look for on that page

Look for	What you'll find
Publisher badge	"Sponsored OSS" — it's official, not a random upload
The <code>docker run</code> example	Your starting command
Internal port	11434
Internal data path	<code>/root/.ollama</code> ← where models are stored
Tags	<code>latest</code> , <code>rocm</code> (AMD), version tags like <code>0.12.3</code>

⚠ Why you actually read the page

Those middle two rows. There is **no way to guess** that a container stores model weights at `/root/.ollama` — and if you don't know it, you can't write the `-v` flag, and you re-download gigabytes on every restart.

③ Create + run Ollama in Docker

```
docker run -d ^
  --name ollama ^
  -p 11434:11434 ^
  -v ollama_data:/root/.ollama ^
  -e OLLAMA_HOST=0.0.0.0 ^
  ollama/ollama
```

💡 Line continuation by shell

Windows CMD: `^` · **PowerShell:** backtick ``` · **Mac/Linux:** `\`

Or just put the whole thing on one line — that always works.

Then pull a model **into** the container:

```
docker exec -it ollama ollama pull llama3.2
docker exec -it ollama ollama list
```

Verify:

```
curl http://localhost:11434
```

Should print: **Ollama is running**

④ Why OpenWebUI is needed

Right now you can only talk to Ollama through `curl` or a terminal. That's fine for testing, painful for actual use.

OpenWebUI gives you a ChatGPT-style browser interface on top of Ollama:

- Chat bubbles instead of terminal text
- **Conversation history saved** — the terminal forgets everything
- Switch models from a dropdown
- Upload files / PDFs for basic RAG
- User accounts
- Edit system prompts without writing a Modelfile

✓ The division of labour

OpenWebUI stores **no models of its own** — it's purely a front end that calls Ollama's API.

Ollama = the engine. OpenWebUI = the dashboard.

⑤ Install OpenWebUI in Docker

It's on **GitHub Container Registry**, not Docker Hub — hence the longer image name:

```
docker run -d ^
  --name openwebui ^
  -p 3000:8080 ^
  -v openwebui_data:/app/backend/data ^
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 ^
  --add-host=host.docker.internal:host-gateway ^
  --restart unless-stopped ^
  ghcr.io/open-webui/open-webui:main
```

⚠ Note the port mapping

-p 3000:8080 — OpenWebUI serves on **8080 inside** the container, you reach it on **3000** outside. The two numbers are independent.

Better: one compose file for both

Create `docker-compose.yml`:

```
services:
  ollama:
    image: ollama/ollama
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    environment:
      - OLLAMA_HOST=0.0.0.0
    restart: unless-stopped

  openwebui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: openwebui
    ports:
      - "3000:8080"
    volumes:
      - openwebui_data:/app/backend/data
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434    # ← service name, NOT localhost
    depends_on:
      - ollama
    restart: unless-stopped

volumes:
  ollama_data:
  openwebui_data:
```

```
docker compose up -d
docker exec -it ollama ollama pull llama3.2
```

⑥ Check both from the browser

URL	Expected
<code>http://localhost:11434</code>	Plain text: Ollama is running
<code>http://localhost:11434/api/tags</code>	JSON list of your models
<code>http://localhost:3000</code>	OpenWebUI login screen

🔥 The most common confusion

If `/api/tags` shows `{"models": []}`, Ollama is **alive but empty** — you haven't pulled a model. This is not a connection failure.

⑦ Create the admin account

1. Open `http://localhost:3000`
2. Click **Sign up**
3. Enter name, email, password — **any email works**. It's stored locally, no verification, no internet.
4. **The first account created automatically becomes admin.**

🔥 There is no password recovery

Lose it and your only option is deleting the `openwebui_data` volume and starting over — which also wipes your chat history.

Later signups become regular users and may need admin approval. Control this in **Admin Panel** → **Settings** → **General**.

⑧ Troubleshooting walkthrough — a real session

This section follows an actual setup, screenshot by screenshot. The symptoms below look like failures and are all **normal**.

Symptom 1 — Green toggle, but is it connected?

In **Admin Panel** → **Settings** → **Connections**, the *Ollama API* toggle is green and shows `http://host.docker.internal:11434`.

 **The green toggle does NOT mean connected**

It only means **"Ollama support is enabled."** That URL is OpenWebUI's **default value** — it's displayed whether or not anything is actually listening there.

The real test

Leave settings, go to **New Chat**, and open the **model dropdown** at the top.

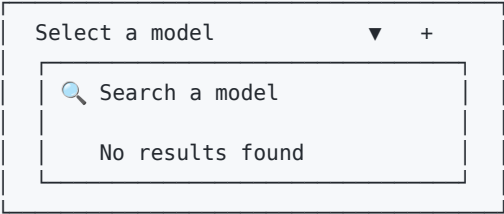
What you see	Meaning
Models listed	Connected, working, done
Empty / "No results found"	Either not connected, <i>or</i> connected with zero models

What that URL tells you about your setup

`host.docker.internal` means *"reach out of this container to the host machine."* That URL only works if **Ollama is running natively**, not in Docker.

Your Ollama setup	Correct URL in that field
Installed natively on Windows (<code>OllamaSetup.exe</code>)	<code>http://host.docker.internal:11434</code>
Running in Docker via compose	<code>http://ollama:11434</code>
Plain <code>docker run</code> , no compose	<code>http://host.docker.internal:11434</code> — needs <code>--add-host</code>

Symptom 2 — "No results found" in the model dropdown



 **This is "engine installed, no fuel"**

An **empty list with a live connection** is exactly what zero models looks like. If the connection were genuinely broken, you'd see an **error toast about reaching Ollama** — not a clean empty list.

The accompanying *"Choose a model before saving..."* warning is just the UI refusing to save a chat with no model selected. Not a connection error.

Symptom 3 — The fix, and the proof

In **Admin Panel** → **Settings** → **Models** → **Manage Models**, type a model name (e.g. `gemma:2b`) and click the download icon.

Pull a model from Ollama.com ↓

gemma:2b

[↓]

gemma:2b

24.9%

sha256:c1864a5eb19305c40519da12cc543519e48a0697ecd30e15d5ac228644957d12

✓ The progress bar IS your connection test — passed

OpenWebUI **cannot trigger a download through Ollama unless it is connected**. A moving progress bar proves the link works. The connection was never the problem.

That `sha256:...` string is the **content-addressed blob ID** from Topic 36 — the actual weight file being fetched into the model store.

After the download completes

1. **Wait for 100%.** `gemma:2b` is ~1.7 GB.
2. **Close** the Manage Models dialog.
3. **Hard-refresh** the browser — `Ctrl+Shift+R`. The model list is cached client-side, so a plain refresh can still show the stale empty list.
4. Open the dropdown → `gemma:2b` appears → select it → chat.

A note on model choice

⚠ Don't judge local LLMs by a 2B model

`gemma:2b` is a fine *first* pull — small and fast. But it's a **2 billion parameter** model, at the bottom of the RAM table from Topic 34. Expect short answers, shaky reasoning, occasional nonsense.

Once it works end to end, pull something in the genuinely useful range:

```
docker exec -it ollama ollama pull llama3.2:3b      # ~2 GB, better quality
docker exec -it ollama ollama pull qwen2.5:7b      # ~4.7 GB, genuinely useful
```

Or use the same OpenWebUI dialog — type the name, click the download icon.

🔗 Back to Topic 36

Pulling **through OpenWebUI** works, but it's a manual step that lives nowhere in your config — exactly the **"Method 1"** trade-off. When you formalise the compose file, move these pulls into an **init container** so `docker compose up` is genuinely one command.

Where your models actually live

```
ollama list      # works → native install
docker ps       # is there an "ollama" container?
```

⚠ Native vs container storage

If Ollama is native on Windows, your models live in `C:\Users\<you>\.ollama` — **not** in a Docker volume. Switch to running Ollama in a container later and you'll **re-download everything**, unless you bind-mount that folder.

🔧 The full diagnostic sequence

Run these three in order when the dropdown is empty:

```
# 1. Is Ollama alive on the host?
curl http://localhost:11434

# 2. Do you have any models at all?
ollama list
# (or, if it's in Docker: docker exec ollama ollama list)

# 3. Can the OpenWebUI container actually reach it?
docker exec openwebui curl -s http://host.docker.internal:11434
```

Result	Fix
#1 fails	Ollama isn't running. Start it.
#2 shows nothing	<code>ollama pull llama3.2</code> — you have the engine but no models
#3 fails but #1 works	Container can't reach the host — recreate with <code>--add-host=host.docker.internal:host-gateway</code>
All three pass, dropdown still empty	Hit the refresh icon next to the URL in Connections, then hard-refresh the browser

📋 Quick reference card

Task	Command / URL
Start everything	<code>docker compose up -d</code>
Pull a model	<code>docker exec -it ollama ollama pull llama3.2</code>
List models on disk	<code>docker exec ollama ollama list</code>
List models in RAM	<code>docker exec ollama ollama ps</code>
Check Ollama is alive	<code>http://localhost:11434</code>
Check models via API	<code>http://localhost:11434/api/tags</code>
Open the chat UI	<code>http://localhost:3000</code>
See what went wrong	<code>docker compose logs -f ollama</code>
Stop everything	<code>docker compose down</code>
Stop and delete models	<code>docker compose down -v</code> ⚠

 MEMORY HOOK

A green toggle proves nothing. A moving progress bar proves everything.

An empty model dropdown almost always means **no models pulled** — not a broken connection.